PROGRAMMING & CS FUNDAMENTALS

Locks & Deadlocks Visual Guide

Mutual exclusion done right —
and the four conditions that freeze systems.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Mutex: One at a Time

Mutual exclusion — the bouncer for critical sections.

```
with lock:                  # acquire on entry, release on exit
    balance += deposit      # critical section: exactly one thread
# RAII / context managers: the lock CANNOT be forgotten
```

- Critical section: code touching shared state — keep it SHORT (contention = serialization).
- Always use scoped acquisition (with/lock_guard) — manual unlock on every path is a bug farm.
- Recursive mutexes allow re-locking by the owner — a design smell; restructure instead.

STEP 2/6

Semaphore: N at a Time

A counter, not a flag — permits for a pool of identical resources.

DB pool: 3 connections. acquire() decrements; 0 -> wait. release() increments.

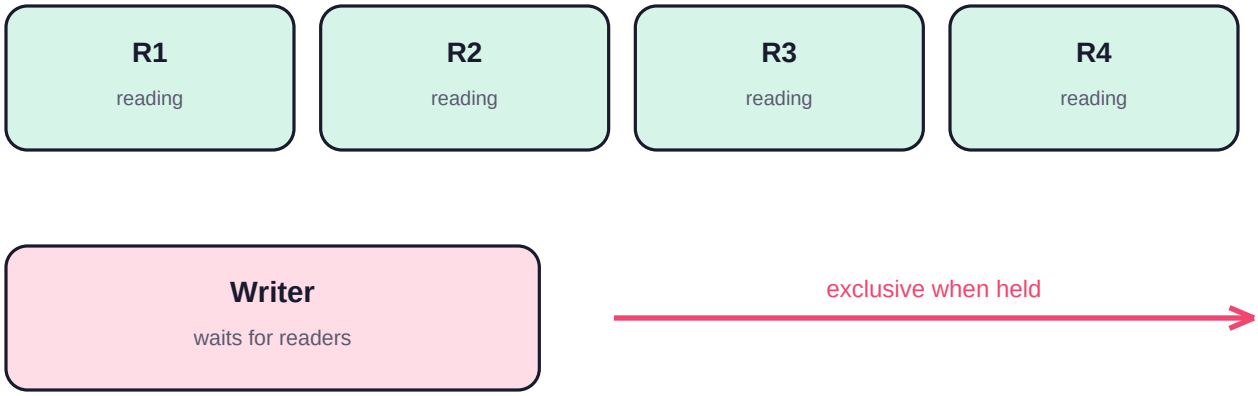
```
sem = Semaphore(3)          # 3 permits
sem.acquire()              # take one (blocks at 0)
use_connection(); sem.release()
# binary semaphore (N=1) ~= mutex, minus ownership tracking
```

- Counting semaphore: resource pools, rate limiters, bounded buffers.
- No owner concept: any thread may release — flexible, and easier to misuse.
- Producer/consumer: one semaphore counts items, another counts free slots.

STEP 3/6

Reader-Writer Locks

Many readers OR one writer — match the lock to the access pattern.



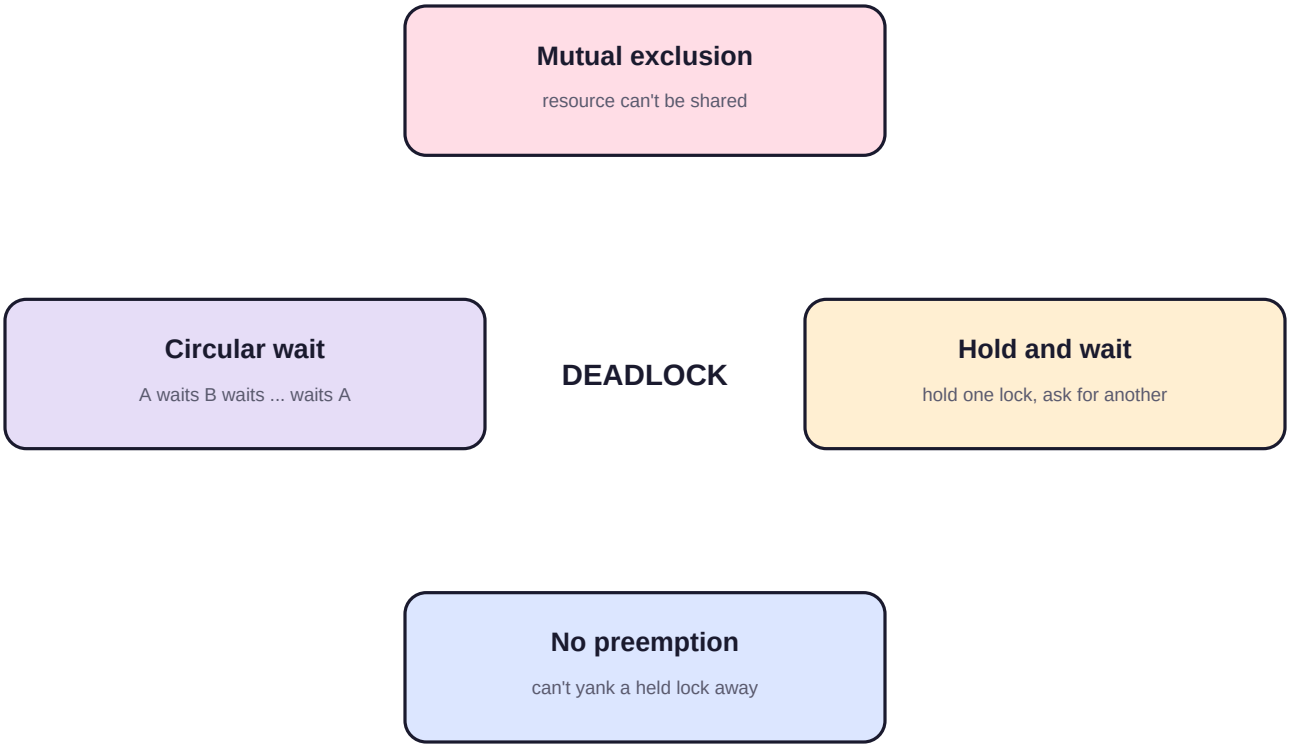

```
with rw.read_lock(): return cache[k]     # readers share freely
with rw.write_lock(): cache[k] = v       # writer excludes everyone
```

- Read-heavy workloads (caches, configs): big win over plain mutex.
- Writer starvation: endless readers can block writers — prefer writer-fair variants.
- Upgrade (read -> write) deadlocks naively — release and re-acquire, or use upgradable locks.

STEP 4/6

Deadlock: The Four Conditions

All four must hold — break any one and deadlock is impossible.



dining philosophers: 5 forks, each holds left, waits right — forever

- Coffman conditions (1971): the checklist every deadlock postmortem uses.
- Detection: lock-order graphs — a cycle in 'waits-for' means deadlock.

STEP 5/6

Prevention vs Avoidance vs Detection

Three strategies, three price tags.

Prevention

break a condition: global lock ORDERING — cheap, sometimes rigid

Avoidance

Banker's algorithm: grant only safe states — costly, needs max claims

Detection

let it happen; find cycles, kill a victim — DBs do this

Ignorance

the ostrich algorithm — most OSes! — reboot if stuck

```
# lock ordering: always acquire in ID order — no cycles possible
first, second = sorted([lock_a, lock_b], key=id)
with first, second: ...
```

Lock ordering is the working engineer's answer: simple, provable, and free. Try-lock with backoff is the pragmatic cousin.

STEP 6/6

Locking Wisdom

Rules from the trenches.

- Hold locks for the SHORTEST time possible — contention serializes your parallel code.
- Never call unknown code (callbacks, virtual methods) while holding a lock — reentrancy deadlock.
- Lock granularity: one big lock (simple, serial) vs many fine locks (parallel, deadlock-prone).
- Prefer lock-free structures (queues, atomics) on hot paths — then measure.
- Document lock order wherever two locks can nest — future you will thank present you.
- Livelock: everyone politely backs off forever — randomize backoff to break symmetry.

STEP ✓/✓

Key Takeaways

Locks & Deadlocks — Remember

- Mutex: one holder; semaphore: N permits; RW-lock: many readers or one writer.
- Deadlock needs all four Coffman conditions — break any one (lock ordering breaks circular wait).
- Strategies: prevention (ordering), avoidance (Banker's), detection (cycle hunt), ostrich.
- Keep critical sections short; never call unknown code while holding a lock.
- Scoped acquisition (with/lock_guard) makes leaks impossible — never hand-unlock.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev